Northeastern University
Khoury College of
Computer Sciences

Anticipate, Simulate, Replan

Egocentric World Models for Multi-Step Agentic Reasoning

Lorenzo Torresani
Northeastern University

Samsung START 2026 · 2.1-233

hanks for the chance to present. I'm Lorenzo Torresani, from Northeastern. I'll spend 15 minutes on the technical proposal, then I'm happy to take questions.

The physical world is a human world

The physical world is, almost everywhere, a human world. Look around any consequential setting — homes, hospitals, factories — and you'll see humans in the loop of every decision. So a critical agenda is the design of AI systems that understand what a person is doing, anticipate where they're going, and step in to help. An assistant that knows when to nudge you mid-task. A surgical AI that has the next instrument ready. AR guidance that adapts to your pace.
Prototypical examples are an assistive robot that knows when to step in on a task the user is struggling with, a surgical AI that anticipates which instrument is needed next, or an AR system that adjusts its guidance to the user's pace and skill.

Doing this well requires two components working together. The first is an egocentric world model — we call it EgoLWM — that watches what's happening and predicts what's about to happen next. The second is a grounding-guided planner — GGPO — that generates multi-step plans grounded in how people actually execute tasks, and revises them online. The proposal is built around these two components and how they couple. Let me show you what each one does.

Model 1: EgoLWM — an egocentric latent world model

Predicts how a first-person scene will evolve, in semantic latent space.

Is this artist about to mix paint, load the brush, or paint a stroke?

Will this person continue to write, or check their phone?

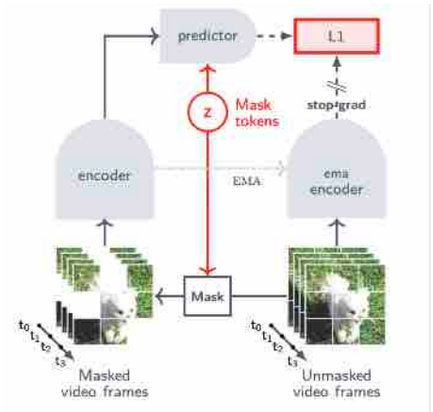
Is this player about to tune, rosin, or play?

EgoLWM's job: anticipate the immediate future from context, and simulate alternative continuations.

For an AI assistant to help you, it has to anticipate what you're about to do next. Three examples. A painter at the easel — about to mix paint, load the brush, or commit to a stroke? A person at their notebook with their phone nearby — about to keep writing, or check the phone? A violinist with the bow in hand — about to tune, apply rosin, or start playing? In all three, the same observed moment admits very different next actions.
EgoLWM is the engine that closes this gap. Given egocentric observation, it predicts how the scene will evolve in the immediate future. Critically, it predicts in semantic latent space, not at the pixel level — so we can simulate alternative continuations cheaply, which is what makes the closed loop tractable.

Model 1: EgoLWM - predict in semantic latent space

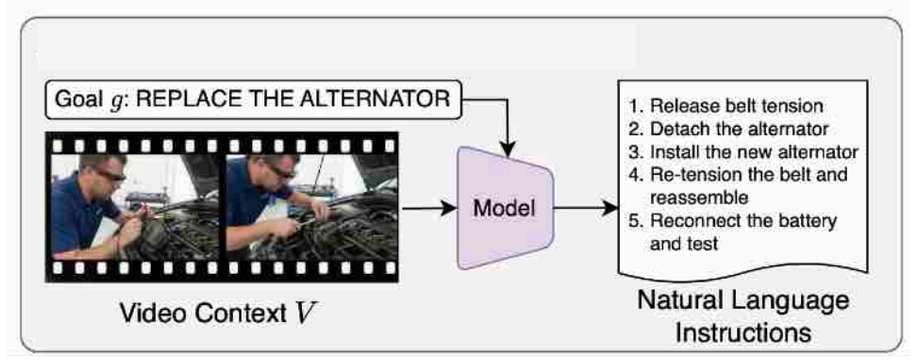
Build on the JEPA line



- **Mask-denoising in representation space.** Encoder + predictor trained jointly on masked video
- **Latent prediction, not pixel generation.** Captures predictable semantic structure; ignores unpredictable detail
- **EgoLWM.** Post-trains this foundation for egocentric multi-step procedural prediction

EgoLWM builds on the V-JEPA line. The idea is to predict in latent space, not pixels — the model learns predictable semantic structure and ignores unpredictable detail. Downstream planning shares the same representation, which is what makes the closed loop tractable. EgoLWM is this foundation post-trained for egocentric multi-step procedural prediction. Now let me show you GGPO.

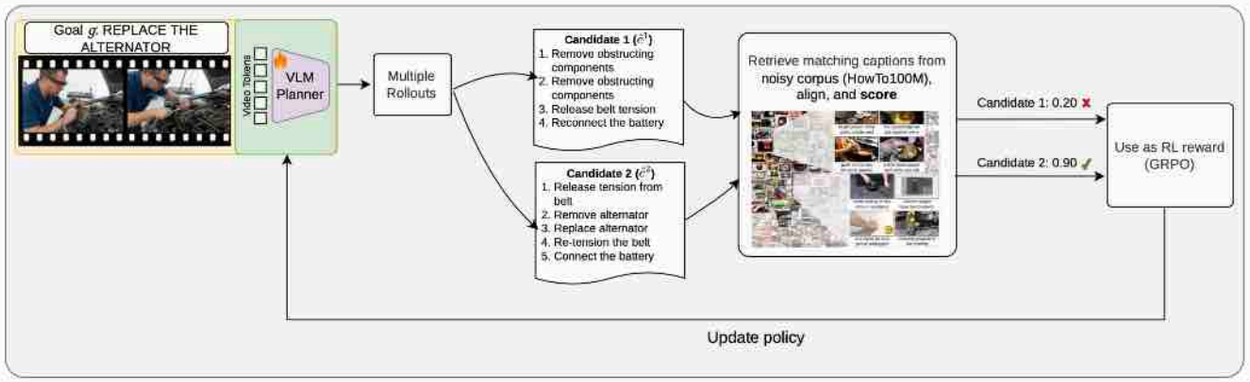
Model 2: GGPO — generating free-form procedural plans



- **Wearable assistant.** Guides a procedural task step-by-step in real time
- **Free-form natural language.** Outputs adapted to tools, parts, and state actually visible in the scene
- **GGPO.** A planning policy trained to generate these multi-step plans

Picture someone wearing smart glasses, trying to replace an alternator for the first time. They need step-by-step guidance — not generic instructions from a manual, but instructions specific to the tools and parts actually on their bench. As they execute, the assistant has to track what's been done, anticipate what's next, and revise if something unexpected happens. The output has to be free-form language, not a closed taxonomy. GGPO is the planning policy that generates these plans. The next slide is the methodological idea that makes it scale.

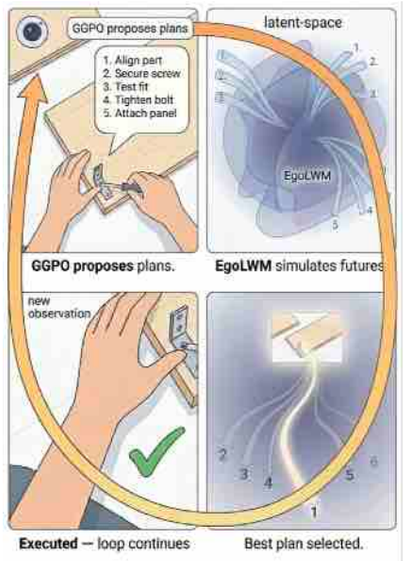
Model 2: GGPO — Grounding-as-verification reward



- **Asymmetry.** Labeling noisy video is hard. Verifying a plan against the corpus is cheap.
- **Reward = grounding score in HowTo100M ASR.**
  Multiple valid orderings all ground. Policy learns plan diversity.
- **Trained with GRPO.** Verification scales where labeling doesn't. No clean step annotations required.

The key idea is to flip the role of large-scale web video. Extracting clean step labels from noisy narration is hard. But asking "does this generated plan make sense against the corpus?" — that's cheap. You can check it with text-only alignment in a precomputed embedding space.
So we use the corpus as a verifier, not a label source. GGPO generates candidate plans; each one earns reward in proportion to how well it grounds in real ASR narrations across the corpus. That score becomes a reward signal for GRPO. The key thing we don't need: clean step annotations.
A useful side effect — multiple valid execution orderings all ground successfully, so the policy learns a distribution over valid plans rather than collapsing onto a single trajectory. That plan diversity is exactly what downstream replanning needs.

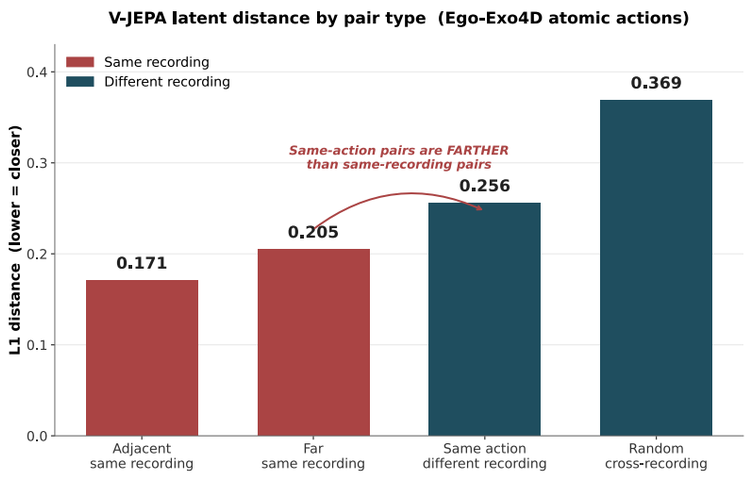
The proposed system: EgoLWM and GGPO coupled in a closed loop



- **GGPO proposes.** Candidate multi-step plans for the current goal.
- **EgoLWM simulates.** Each plan rolled forward in latent space, scored against the goal latent.
- **Execute and revise on surprise.** Observed-vs-predicted divergence triggers GGPO to replan online.

Putting the two modules together. The system runs in a closed loop with three operations at each decision step.
Anticipate. GGPO proposes a small set of candidate plans for the current goal — concretely, the assembly steps for the bracket on the screen.
Simulate. EgoLWM rolls each candidate forward in its latent space and scores the simulated trajectories against the goal latent. Because prediction and planning share a representation, no pixel decoding is needed.
Execute and revise. The highest-scoring plan is executed. After execution, the observed latent trajectory is compared to the predicted one. A divergence exceeding a calibrated threshold — a surprise signal — triggers GGPO to revise the remaining plan online.
Before locking in the EgoLWM design that this loop depends on, we ran two preliminary diagnostics that sharpened the specification. Those are the next two slides.

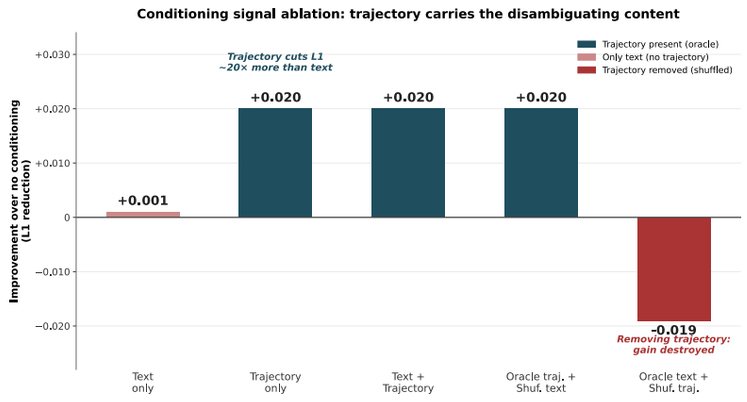
Diagnostic 1: V-JEPA latent space is not a planning space



- **Visual continuity, not action.** Same-recording > same-action.
- **Distance-to-goal misled.** Picks up recording, not action.
- **LeWM same.** Both optimize for continuity.

Diagnostic one is a direct probe of latent geometry. We took V-JEPA 2 features on Ego-Exo4D atomic-action segments and asked which pairs of clips are close in this latent space.
The plot shows the answer. Red bars are same-recording pairs — adjacent clips and far clips from the same continuous recording. Teal bars are different-recording pairs. The asymmetry: same-recording proximity is closer than same-action proximity. The arrow points at the inversion — clips of the same action drawn from different recordings sit farther in latent space than unrelated clips from the same recording.
The implication is structural: V-JEPA latents organize by visual continuity, not action semantics. Any planning method that scores candidates by distance-to-goal — the natural baseline for latent-space planning — is structurally misled by this geometry.
One important caveat. This isn't a V-JEPA 2 specific issue. LeWM has the same property because it also optimizes for visual continuity. So switching encoders within the visual-continuity family doesn't fix it.

Diagnostic 2: trajectory dominates language as conditioning signal



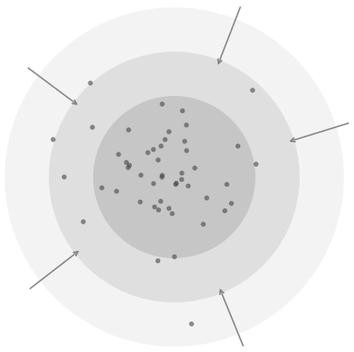
- **Trajectory >> language.**
  Trajectory cuts L1 ~20× more.
- **Text adds nothing on top.**
  Text + traj = traj alone.
- **Shuffle ablations confirm.**
  Trajectory carries the signal.

Diagnostic two — which signal carries the per-step disambiguating content? We trained predictors under five conditions.
Two patterns. First — text alone barely helps. Trajectory alone improves L1 twenty times more. Adding text on top of trajectory adds nothing.
Second — the shuffle ablations confirm. The fourth bar — oracle trajectory plus shuffled text — gives the same gain as trajectory alone, because text wasn't carrying useful signal anyway. The fifth bar — oracle text plus shuffled trajectory — destroys the gain. Removing trajectory matters; text does not.
Implication: at the per-step level, trajectory is the right input.

Hybrid regularizer:
re-targeting LeWM toward an action-aligned latent geometry

LeWM target

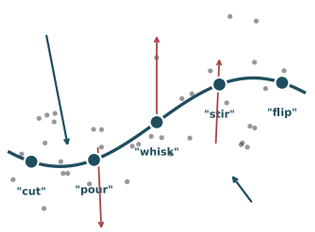
Isotropic Gaussian prior



SIGReg pushes latents toward an isotropic prior (prevents collapse — but no action structure)

EgoLWM target

Action-aligned manifold (this proposal)

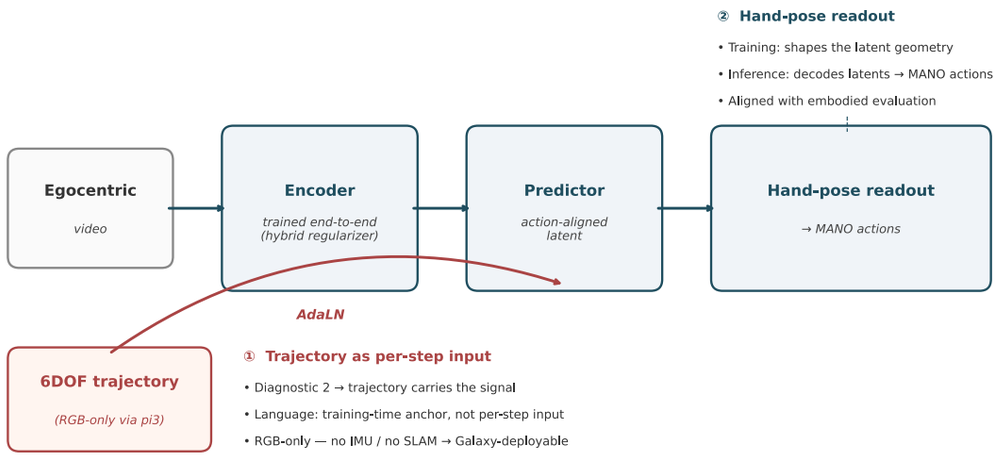


Pull toward action manifold (teal) + anti-collapse perpendicular to it (red)

Non-trivial design: SIGReg is constructed for the isotropic-Gaussian setting and does not transfer directly.

The two diagnostics gave us a clear specification: end-to-end training is required, and the latent target must be organized by action semantics. The challenge is how to deliver that without losing the stability that makes JEPA training work.
Start from LeWM. LeWM gives a stable end-to-end JEPA training recipe via a two-term objective: predict the next latent, plus SIGReg, a regularizer that pushes latents toward an isotropic Gaussian distribution to prevent collapse. The left picture: latents pushed toward a structureless prior. Stable, but no action structure.
EgoLWM re-targets the latent. Instead of an isotropic prior, we want a target distribution defined by language-grounded action embeddings — the labeled dots on the right.
Designing the regularizer that achieves this is the non-trivial part. SIGReg is constructed specifically for the isotropic-Gaussian setting, so it doesn't transfer directly. Our plan is a hybrid that pulls latents toward the action manifold along action-aligned directions — the teal arrows — while preserving SIGReg-style anti-collapse on directions orthogonal to the manifold — the red arrows. The orthogonal subspace still needs anti-collapse protection, and that's where SIGReg's machinery still applies.
This is the work proposed for Q1 of Year 1.

Closed-loop integration: Anticipate, Simulate, Replan



Two design choices:

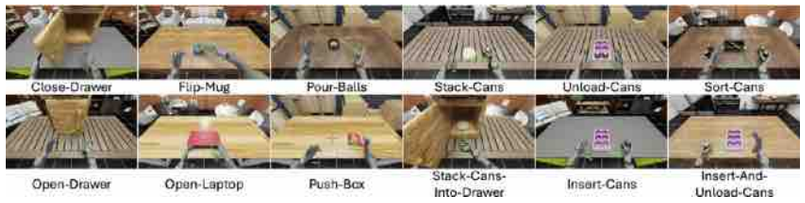
① trajectory delivers per-step disambiguation (diagnostic 2)

② hand-pose aligns training with the embodied evaluation

Two design choices beyond the regularizer. Both are driven by what we covered earlier.
First — trajectory as per-step input, injected via adaptive layer normalization at every step. This comes directly from diagnostic two: trajectory carries the disambiguating content, language doesn't. So language gets demoted from per-step input to training-time anchor for the latent target. One practical point: the trajectory is estimated from RGB video alone via pi3 — no IMU, no SLAM. That keeps the system deployable across the full Galaxy device range.
Second — a hand-pose readout head that decodes predicted latents into MANO actions. Dual role. At training, the readout shapes the latent geometry using Ego-Exo4D's 14 million frame hand annotations. At inference, the same head decodes predicted latents into actions for the embodied evaluation on the next slide.

Demonstration + Timeline + Samsung

- **Embodied demonstration**



Ego Humanoid Manipulation Benchmark
12 bimanual tasks on Unitree H1
vs. EgoVLA · vs. reactive (ours)

- **Year-1 timeline**

Q1	Q2	Q3	Q4
Sep-Nov 2026	*Dec 2026 - Feb 2027*	*Mar-May 2027*	*Jun-Aug 2027*
Hybrid regularizer	**Architectural ablations**	**Procedural benchmarks**	**Closed-loop integration**
+ end-to-end pipeline	+ trajectory predictor	+ embodied eval begins	+ protocol release

- **Year 2/3 + Samsung**

Year 2: Digital and UI bridge. Screen + cursor replace ego-video.

Year 3: Deployment readiness. Latency, memory, on-device integration.

Three things to close on.
First — end-to-end embodied validation, explicitly requested in the review feedback. We use the Ego Humanoid Manipulation Benchmark introduced with EgoVLA — Isaac Lab, Unitree H1 humanoid with two Inspire hands, twelve bimanual tasks across seen and unseen visual configurations. The action space is MANO, matching our hand-pose readout exactly. The critical baseline is a reactive variant of our own system, which directly quantifies the value of the closed-loop architecture.
Second — Year 1 timeline. Four quarters with explicit gates: hybrid regularizer in Q1, architectural ablations and trajectory predictor in Q2, procedural benchmarks and embodied evaluation begins in Q3, closed-loop integration and protocol release in Q4.
Third — Samsung integration. Year 2 extends the architecture from physical to digital and UI tasks for the on-device Galaxy AI scenario. Year 3 is deployment readiness with Samsung partners. Three reasons this fits: latent prediction is orders of magnitude faster than pixel diffusion, enabling real-time replan on wearables; RGB-only trajectories mean no specialized hardware; and the architecture sits downstream of Samsung's existing streaming-video stack.
Thank you. Happy to take questions.